Python Cheatsheet

A compact reference with concise explanations, practical examples and tips about common mistakes.

Some examples are code fragments rather than standalone programs; they use variables or inputs described in the text.

Variables and Types

Basic values and assignment.

```
name = "Anna"
age = 21
active = True
value = None
```

Conditionals

Branching based on a Boolean condition.

```
if score >= 50:
    status = "passed"
elif score >= 40:
    status = "retake"
else:
    status = "failed"
```

for Loop

Iteration over items in a collection.

```
for value in values:
    print(value)
```

while Loop

Repeat while a condition remains true.

```
while attempts < 3:
    attempts += 1
```

range()

Numeric sequences without creating a list.

```
range(5)        # 0, 1, 2, 3, 4
range(2, 8)     # 2, 3, 4, 5, 6, 7
range(0, 10, 2) # 0, 2, 4, 6, 8
```

list

Mutable ordered collection.

```
names = ["Anna", "Peter"]
names.append("Lucia")
first = names[0]
```

Tip

sort() changes the original list and returns None. Use sorted(values) when you need a new sorted list.

tuple

Immutable ordered collection.

```
record = ("Anna", 82, "Python")
name, points, course = record
```

set

Collection of unique values.

```
courses = {"Python", "Databázy"}
courses.add("Testovanie")
```

dict

Mapping from key to value.

```
points = {"Anna": 92, "Peter": 67}
points["Lucia"] = 78

for name, score in points.items():
    print(name, score)
```

Tip

Accessing a missing key raises KeyError. Use get() for an optional key.

Indexing and Slicing

Access items and slices of a sequence.

```
values[0]
values[-1]
values[1:4]
values[:3]
values[::2]
values[::-1]
```

Tip

The stop index is excluded. For example, values[1:4] returns items at indexes 1, 2 and 3.

Membership: in

Check whether a value is present.

```
if value in values:
    print("found")

if "Python" not in courses:
    courses.add("Python")
```

enumerate()

Index and value during iteration.

```
for index, value in enumerate(values):
    print(index, value)
```

Tip

When you need both index and value, prefer enumerate() over range(len(values)).

zip()

Parallel iteration over multiple collections.

```
for name, score in zip(names, scores):
    print(name, score)
```

Tip

zip() stops at the shortest iterable by default. Check that input collections have the expected lengths.

Unpacking

Unpack values into variables.

```
first, second = pair
name, *middle, last = values
```

Comprehensions

Compact syntax for creating collections.

```
squares = [n ** 2 for n in numbers]
positive = [n ** 2 for n in numbers if n > 0]
unique_courses = {course for _, course in records}
points_by_name = {name: points for name, points in rows}
```

Tip

Use a comprehension for a simple transformation or filter. If it needs many conditions or steps, a regular loop may be clearer.

Functions

Named unit of logic with a return value.

```
def average(values):
    return sum(values) / len(values)
```

Tip

Decide how the function should handle an empty list. Without a check, division by zero raises ZeroDivisionError.

Function Arguments

Default and keyword arguments.

```
def greet(name, greeting="Ahoj"):
    return f"{greeting}, {name}"

greet("Anna")
greet(name="Peter", greeting="Dobré ráno")
```

Tip

Avoid mutable default arguments such as def add(item, items=[]). The same list may then be shared between calls.

Type Hints

Types as part of a readable API.

```
def passed(points: int, threshold: int = 50) -> bool:
    return points >= threshold
```

Tip

Type hints improve documentation and IDE support, but do not validate types at runtime by themselves.

dataclass

Simple data structure.

```
from dataclasses import dataclass

@dataclass
class Result:
    student: str
    course: str
    points: int
```

Exceptions

Controlled exception handling.

```
user_input = input("Počet bodov: ")
try:
    points = int(user_input)
except ValueError:
    print("Zadaj celé číslo.")
else:
    print(f"Zadané body: {points}")
```

Tip

Do not silently turn invalid input into 0 when 0 has a real meaning. Report the error or ask for input again as appropriate.

Imports

Use modules and specific objects.

```
import math
from pathlib import Path
from dataclasses import dataclass
```

Tip

Avoid from module import *. It makes it unclear where names come from.

String Methods

Common text operations.

```
text.strip()
text.split(",")
text.lower()
text.replace("old", "new")
text.startswith("ERR")
text.endswith(".py")
```

Tip

Strings are immutable. replace() returns a new string; assign the result if you need to keep it.

Common Patterns

Short idioms that appear frequently in Python code.

```
if value is None:
    ...

for key, value in data.items():
    ...

if item in collection:
    ...
```

Classes and Objects (OOP)

A class describes shared object structure and behavior; an object is a particular instance of a class.

```
class Student:
    def __init__(self, name, points):
        self.name = name
        self.points = points

    def passed(self, threshold=50):
        return self.points >= threshold

student = Student("Anna", 72)
print(student.passed())
```

Tip

self refers to the current instance. Python passes it automatically when you call student.passed().

pathlib

Work with paths and files without manually joining path strings.

```
from pathlib import Path

folder = Path("data")
file_path = folder / "notes.txt"

if file_path.exists():
    text = file_path.read_text(encoding="utf-8")
```

Tip

Use Path instead of manually joining path strings. Specify an encoding for text files.

json

Convert between Python data and the JSON text format.

```
import json

student = {"name": "Anna", "points": 92}
text = json.dumps(student, ensure_ascii=False)
restored = json.loads(text)
```

Tip

loads() parses text and load() reads a file object; dumps() creates text and dump() writes to a file object.

datetime

Work with dates, times and time differences without manually parsing strings.

```
from datetime import date, datetime

today = date.today()
now = datetime.now()
print(today.isoformat())
print(now.strftime("%Y-%m-%d %H:%M"))
```

Tip

For timezone-dependent work, consider timezone-aware datetimes; datetime.now() alone returns local time without an explicit timezone.

collections.Counter

Count value occurrences without maintaining counters manually.

```
from collections import Counter

votes = ["A", "B", "A", "C", "A"]
counts = Counter(votes)
print(counts["A"])  # 3
print(counts.most_common(2))
```

Tip

Counter is useful for frequencies. If you only need unique values, a set may be enough.

re

Search and validate text patterns with regular expressions.

```
import re

text = "Order number: 4582"
match = re.search(r"\d+", text)
if match:
    order_number = int(match.group())
```

Tip

Use regular expressions for text patterns, not as a substitute for a parser for complex formats.

Built-in Functions

Functions available without an import. Common uses at a glance:

`len(x)` Number of items.	**`min(x) / max(x)`** Smallest / largest value.
`sum(x)` Sum of numeric values.	**`sorted(x)`** Returns a new sorted list.
`enumerate(x)` Index and value during iteration.	**`zip(a, b)`** Pairs items from multiple iterables.
`range(n)` Numeric sequence.	**`any(x) / all(x)`** Test whether any / all values are truthy.
`type(x)` Returns the object's type.	

Tip

sorted() returns a new list; list.sort() changes the existing list.